

Tabla de Símbolos

Diseño e Implementación de un Lenguaje

v0.1.0

1. Introducción **1**

1. Introducción

Toda aplicación requiere de cómputo y almacenamiento (esto debido a que al menos requiere de un lugar para almacenar su código fuente). En un compilador cualquiera, el almacenamiento puede estar constituido por múltiples estructuras de datos, pero una de aquellas que posee mayor relevancia, y una fuerte relación con todas las fases del mismo, es la **tabla de símbolos**.

La tabla de símbolos (que no necesariamente debe ser una tabla), representa la base de datos del compilador, y es el lugar preferido para depositar información acerca de las construcciones del lenguaje diseñado, en particular de aquellas que abarquen múltiples fases (por fases se entienden a las distintas etapas del compilador, análisis léxico, sintáctico, semántico, etc.).

En general, la tabla de símbolos podría contener (entre otras cosas):

- Una entrada por cada variable o constante declarada.
- Una entrada por cada función o método declarado.
- Una entrada por cada definición de alto nivel, como una clase, estructura, módulo, etc.

Es posible que, dependiendo del lenguaje diseñado, sea necesario incluir más información en dicha estructura. Por ejemplo, para un lenguaje donde es posible definir redes de comunicación, pokemones, autos, etc., es probable que dichas abstracciones también inserten información en la tabla.

La necesidad de la tabla de símbolos radica en que es relevante para cada fase del compilador, no solo comprender la estructura del programa (mediante el AST), sino también, tener certeza acerca de la forma en la que el *estado del programa* evoluciona a lo largo del mismo.

Por ende, es importante lograr determinar en cada parte del programa (es decir, en cada nodo del AST), qué cantidades, variables, métodos, funciones, clases, etc., se encuentran activas y con qué características.

En principio, y suponiendo que se desea crear una tabla para el siguiente programa:

```
00 int x;
01 float y = 3.14f;
02 for (unsigned int i = 0; i < 5; ++i) {
03     x = 2 * i;
04     y += x;
05 }
06 double z = 48 * u;
07 char x = '\t';
```

Para el programa, las variables de interés son *x*, *y*, *z*, *i* y *u*. Inicialmente, sería posible considerar que es necesario conocer el tipo de dato de cada variable, y además, saber qué variables realmente fueron declaradas e inicializadas correctamente. Para ello, se construirá la siguiente versión (básica), de una tabla de símbolos:

Id	Type	Initialization
<empty>	<empty>	<empty>

El *lexer* (analizador léxico), comenzará a escanear el programa de entrada. Suponiendo que todos los identificadores de variables sean representados mediante el token ID, la línea `00` emitirá uno de ellos para el lexema *x*. La tabla se completa:

Id	Type	Initialization
x	<empty>	<empty>

Si bien el lexema *int* ya fue visto por el lexer, no es posible asignarlo en la tabla de símbolos ya que no se sabe a qué variable pertenece desde ese punto de vista. Al ver el token ID para el lexema x, es posible crear una nueva entrada, en particular porque la tabla no contiene una para este identificador (es decir, se crea una entrada por cada nuevo token ID).

Cuando el *parser* (analizador sintáctico), ejecute la regla de producción asociada a la construcción de la línea `00` (una declaración de variables), entonces obtendrá todo el contexto que necesita para asociar el token ID con el tipo de dato (suponiendo que se encuentra representado por el token TYPE):

Id	Type	Initialization
x	int	<empty>

Como la regla de producción para la línea `00` ya se ejecutó (junto con su acción), nunca más se volverán a analizar sus lexemas y tokens; por suerte la tabla de símbolos ya contiene toda la información que podía extraerse de dicha construcción.

Para la línea `01` se procede de forma análoga, primero identificando el lexema *y*, lo que crea una nueva entrada en la tabla (ya que es un nuevo token ID). Al ejecutar la regla de producción asociada (una inicialización), se identifica además el tipo de dato, y un valor inicial:

Id	Type	Initialization
x	int	<empty>
y	float	3.14

Para la línea `02`, se dan 3 construcciones dentro del bucle *for*. La primera parte declara una nueva variable *i* inicializada:

Id	Type	Initialization
x	int	<empty>
y	float	3.14
i	unsigned int	0

La segunda es una condición, $i < 5$, la cual no declara ninguna variable, pero utiliza (referencia) una que debe haber sido declarada previamente. En este momento, el lexer (quien ve por primera vez el token ID), busca en la tabla de símbolos el lexema *i*, esta vez con éxito, por lo que no genera una entrada adicional en la misma.

Cuando la regla de producción identifica la condición, entonces es momento de verificar la existencia de la misma, ya que no se trata de una declaración. Debido a que hay una entrada en la tabla para el lexema *i*, la variable existe, y no es necesario emitir un error de compilación (*e.g.*, "*Error: undefined variable 'i' in line 2*"). De forma análoga, al ver la sentencia ++i, se procede de la misma forma: ni se crea una entrada en la tabla, ni se emite un error.

Para la línea `03`, `04` y `05` se procede de la misma manera, salvo que en las primeras dos, se realizan dos búsquedas en la tabla, mientras que para la línea `05` no se realiza ninguna consulta.

La línea `06` declara una nueva variable:

Id	Type	Initialization
x	int	<empty>
y	float	3.14
i	unsigned int	0
z	<empty>	<empty>

La definición se separa en dos reglas de producción. La primera es una inicialización de la forma:

initialization ⟶ TYPE ID = EXPRESSION

La segunda, es una *expresión*, representada en este caso por la secuencia 48 * *u*. Si bien la construcción se escanea de izquierda a derecha, la primera regla en reducirse es la de la última secuencia. Sin embargo esta secuencia es una expresión y no una declaración, con lo cual se espera que el ID *u* se encuentre en la tabla de símbolos, cosa que no ocurre. Es momento de emitir un error, ya que claramente se está en presencia de una variable no declarada con anterioridad.

Suponiendo que el compilador continúe a pesar de emitir el error (lo puede hacer, ya que si bien la variable no fue declarada, la estructura sintáctica del programa es correcta), logrará identificar el tipo de la variable *z*:

Id	Type	Initialization
x	int	<empty>
y	float	3.14
i	unsigned int	0
z	double	<empty>

No carga ninguna inicialización ya que la misma depende de una expresión y no de un valor constante como en algunos de los casos anteriores.

Para la línea `07` se produce un error contrario al anterior: la variable *x* referenciada existe, pero se trata de una declaración y no de una construcción de uso, con lo cual se está intentando **re-declarar** un identificador. El lexer busca en la tabla el ID *x*, lo encuentra e ignora (no crea una nueva entrada). Cuando el parser aplica la reducción de la declaración de variables, determina nuevamente que el ID involucrado es *x*, pero como este ya existe en la tabla, emite un error acorde.

Es importante notar que la tabla se puede extender tanto como sea deseado y necesario para el lenguaje desarrollado. Por ejemplo, para simplificar la generación de código, es posible agregar una columna con algún identificador único *auto-generado* que permita su uso en lugar del nombre original que poseía en el programa de entrada que está siendo compilado. Otro uso posible se da al ingresar entradas de construcciones más complejas, como la de una función o clase, en cuyo caso quizás sea deseable ingresar referencias hacia parámetros y propiedades, con el objeto de mantener la información relacionada de alguna manera.

Ya que la búsqueda se realiza por lexema, particularmente por el que provee el token ID, es usual implementar una tabla de símbolos mediante un mapa asociativo, diccionario o *hash-table*, indexado por ID. No obstante, aunque ineficiente, es posible utilizar una simple lista enlazada o un vector dinámico.

¿Qué operaciones debe definir esta base de datos para su uso a través de las fases del compilador? Una de **creación** (al ver un nuevo token ID), una de **actualización** (al ver el tipo de dato de cierto ID, su inicialización o cualquier otra propiedad de interés que deba almacenarse en la tabla), y una de **lectura** (al consultar las propiedades o existencia de cierto ID).